

END-TERM PROJECT-02 REPORT

ON

<Flappy Bird Game>

B.Tech. [Computer Science and Engineering]

Semester: 6th

Section: 23412F3

Course Title & Code: Game Programming with C++ (CSE 3545)

Submitted By:

Student Name: Amulya Shrivastava

Registration No: 2341019456

Guided By: Mr. Sougata Sheet

Academic Year :2025-2026



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
FACULTY OF ENGINEERING AND TECHNOLOGY, INSTITUTE OF
TECHNICAL EDUCATION AND RESEARCH
SIKSHA 'O' ANUSANDHAN (DEEMED TO BE) UNIVERSITY
BHUBANESWAR, ODISHA, INDIA

ABSTRACT

This project focuses on the design and development of a simple Flappy Bird-style game implemented using C++ and the SFML (Simple and Fast Multimedia Library). The main objective of this project is to create an interactive 2D game that demonstrates real-time rendering, event handling, and object-oriented programming concepts. The project also serves as a practical application of programming fundamentals by combining graphics, animation, and logic into a complete gaming experience. It provides an effective platform for understanding how multimedia libraries can be integrated with C++ to build engaging and responsive applications.

The game allows the player to control a bird that must navigate through moving pipes without colliding. The score increases as the player successfully passes obstacles, and a high score is maintained using file handling. This project enhances understanding of game loops, collision detection, graphics rendering, and user interaction. In addition, the game incorporates smooth animations, keyboard-based controls, and dynamic obstacle generation to improve the overall gameplay experience. The implementation also highlights the importance of efficient memory management and structured code organization in game development.

Overall, this project demonstrates the practical use of object-oriented programming techniques in developing a real-time 2D game environment. It combines creativity with technical knowledge to create an entertaining application while strengthening concepts such as event-driven programming, data management, and graphical user interface design. The project can further be extended by adding features such as sound effects, multiple difficulty levels, enhanced graphics, and multiplayer support, making it a strong foundation for advanced game development projects.

Amulya Shrivastava

Submitted By

Mr. Sougata Sheet

Guided By

TABLE OF CONTENTS

Section No.	Topic	Page No.
01	Introduction	01
02	Problem Statement	02
03	Objectives of the Project	03
04	Tools and Technologies Used	04
05	Methodology	05
06	Results Analysis and Screenshots	06 – 08
07	Logical Understanding	09
08	Conclusion	10
09	Future Scopes and Application	11
	References	12
	Annexure 1	13 – 28

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Figure 1	Start Screen For Project	06
Figure 2	Game Screen For Project	07
Figure 3	Game Screen For Project	07
Figure 4	Game Screen For Project	07
Figure 5	Game Screen For Project	08
Figure 6	Game Screen For Project	08

01. Introduction

This project presents the development of a 2D Flappy Bird-inspired game using C++ and the SFML (Simple and Fast Multimedia Library) framework. The game is designed around a player-controlled bird that must continuously fly through gaps between moving pipes while avoiding collisions and achieving the highest possible score. The implementation focuses on creating an interactive and engaging gaming experience through smooth graphics rendering, real-time keyboard interaction, collision detection, score management, and continuous object movement. The project also demonstrates the effective use of the SFML library for handling graphics, audio, animation, and event management in a real-time game environment.

The development of the game emphasizes the practical application of Object-Oriented Programming concepts such as classes, inheritance, encapsulation, abstraction, and polymorphism. A modular and organized code structure is used to improve maintainability, scalability, and code reusability. Features such as responsive controls, sound integration, score tracking, and smooth gameplay mechanics contribute to an enhanced user experience. The project also highlights important game development concepts including game loops, event handling, collision detection, and frame-based updates required for efficient real-time application performance.

The game was successfully implemented and tested under different gameplay conditions to ensure stability, smooth execution, and proper functionality. Various debugging and optimization techniques were applied to improve performance and reduce errors during gameplay. The project provided valuable practical exposure to multimedia programming and software development using C++ and SFML while improving logical thinking, problem-solving abilities, and understanding of interactive application design. Overall, the project demonstrates how object-oriented programming principles and multimedia libraries can be effectively combined to develop an engaging and efficient 2D gaming application.

02. Problem Statement

The problem addressed in this project is to design and implement a real-time 2D Flappy Bird-inspired game using C++ and the SFML graphics library that provides smooth gameplay, responsive controls, and efficient game mechanics. The project focuses on developing an interactive gaming application where the player controls a bird that continuously falls due to gravity and must avoid colliding with moving obstacles in the form of pipes. The game must maintain proper object movement, accurate collision detection, score tracking, and smooth rendering while ensuring stable real-time performance. The project also aims to demonstrate how multimedia programming and object-oriented design techniques can be combined to create an engaging and efficient gaming application.

The game is designed to satisfy the following requirements:

- A bird continuously falls due to gravity and requires constant player interaction.
- The player must control the bird's movement to avoid obstacles (pipes).
- Smooth bird movement and responsive keyboard controls.
- Continuous generation and movement of obstacles during gameplay.
- Accurate collision detection between the bird and pipes.
- Real-time score tracking and display system.
- Storage and management of the highest score achieved.
- Efficient rendering using the SFML graphics library.
- Proper game loop management and frame-based updates.
- Use of Object-Oriented Programming concepts for modular game design.

The program is developed using programming concepts such as conditional statements, loops, functions, classes, and event handling mechanisms. The challenge lies in implementing smooth gameplay mechanics, gravity-based movement, accurate collision handling, and persistent score storage without affecting performance. The project demonstrates how C++ and SFML can be used to create simple yet effective multimedia applications with interactive graphics and real-time user interaction. Overall, the project provides valuable practical experience in game development, logical problem-solving, and object-oriented programming techniques.

03. Objectives of the Project

The objectives of this project are defined to provide a clear direction for the design and development of the Flappy Bird Game while ensuring smooth gameplay, efficient functionality, and proper implementation of real-time game development concepts. The project focuses on combining programming logic, multimedia handling, and user interaction using C++ and the SFML library. It also aims to improve understanding of object-oriented programming principles, graphics rendering, event handling, and collision management through practical implementation. The project helps in enhancing logical thinking, debugging abilities, and overall programming confidence through the development of an interactive gaming application.

The objectives of this practical project are:

- To develop a graphical game using C++ and the SFML multimedia library
- To implement object-oriented concepts such as classes, encapsulation, inheritance, and modular programming
- To simulate real-time motion and gravity effects using continuous game loops
- To implement collision detection logic between the bird and obstacles
- To maintain and update high scores using file handling techniques
- To create smooth gameplay with responsive keyboard controls and animations
 - To understand event handling and frame-based updates in real-time applications
- To enhance debugging, logical thinking, and problem-solving skills
- To gain practical exposure to multimedia programming and real-time software applications
- To improve knowledge of game development concepts such as object movement, score management, and obstacle generation

Overall, the project is designed to provide practical experience in developing interactive multimedia applications using C++ and SFML while strengthening programming and software design skills.

04. Tools and Technologies Used

The following tools and technologies were used in Project 1 and Project 2:

- Programming Language: C++ (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
- Graphics and Audio Library: SFML (Simple and Fast Multimedia Library)
- Compiler: g++ (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
- Development Environment: Visual Studio Code / Terminal-Based Compilation
- Operating System: Ubuntu-24.04.3 LTS

These tools provide the required environment for rendering sprites, handling keyboard input, playing sounds, displaying HUD text, and managing the real-time game loop.

05. Methodology

The methodology followed during the development of this project includes:

1. Requirement analysis:

Identified all important game features such as gravity-based movement, bird flapping, obstacle generation, collision detection, score management, high-score storage, difficulty settings, sound effects, and restart functionality.

2. Class design:

Developed a Bird class for player movement and physics, a Pipes class for obstacle generation, and a Game class for handling rendering, scoring, collisions, game states, and asset management.

3. Asset loading:

Loaded all necessary bird sprites, pipe textures, fonts, and audio files from the Assets folder to support smooth graphics and sound integration.

4. Game loop:

Implemented the real-time game loop using SFML events, frame-based updates, rendering functions, and game state management for responsive gameplay.

5. Pipe system:

Created a pipe spawning mechanism where pipe pairs appear from the right side with randomized gap positions and difficulty-dependent spacing.

6. Difficulty system:

Implemented Easy, Medium, and Hard modes with different pipe speeds, gap sizes, and oscillation movements to gradually increase gameplay challenge.

7. Collision and scoring:

Applied SFML FloatRect boundaries to detect collisions with pipes, the top boundary, and the ground while increasing the score whenever the bird successfully crossed a pipe pair.

8. File handling:

Used HighScore.txt file handling techniques to store and retrieve the highest score achieved during gameplay sessions.

06. Results Analysis and Screenshots

This section describes the results obtained after implementing the C++ program.

The program was successfully executed and produced the expected output. It demonstrates the correct use of programming concepts and provides accurate results based on user input.

- Smooth bird movement with gravity simulation
- Pipes generated dynamically
- Collision detection works correctly
- Score increments properly
- High score persists across sessions using file

The gameplay performance remained stable during execution and provided responsive user interaction.

The project successfully delivers a fully functional and playable version of the classic Flappy Bird game, featuring smooth gameplay mechanics, responsive controls, and an engaging user experience.

Figure 1: Flappy Bird Game Window (Start Window)

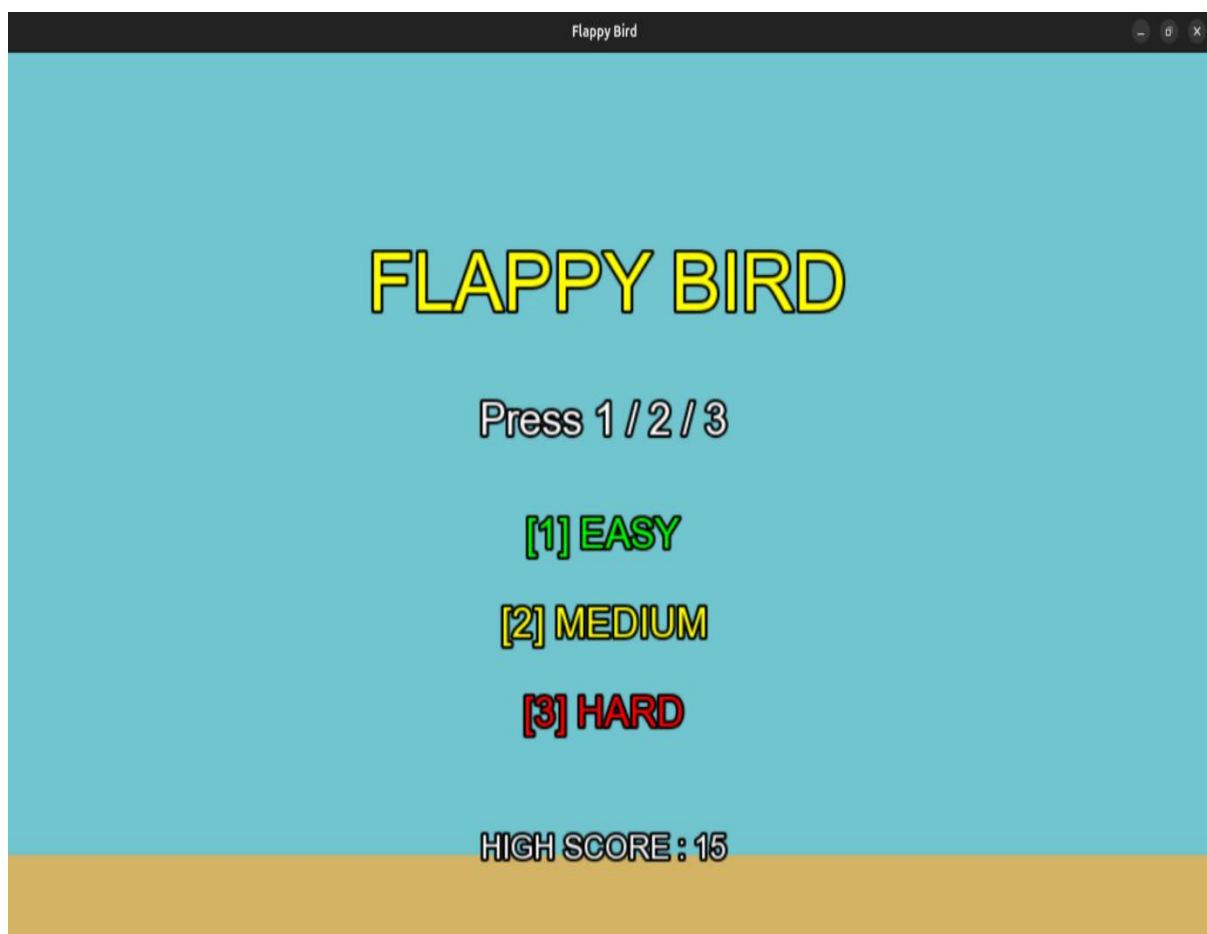


Figure 2: Easy Mode Game Play (Space Is Being Used As Control)

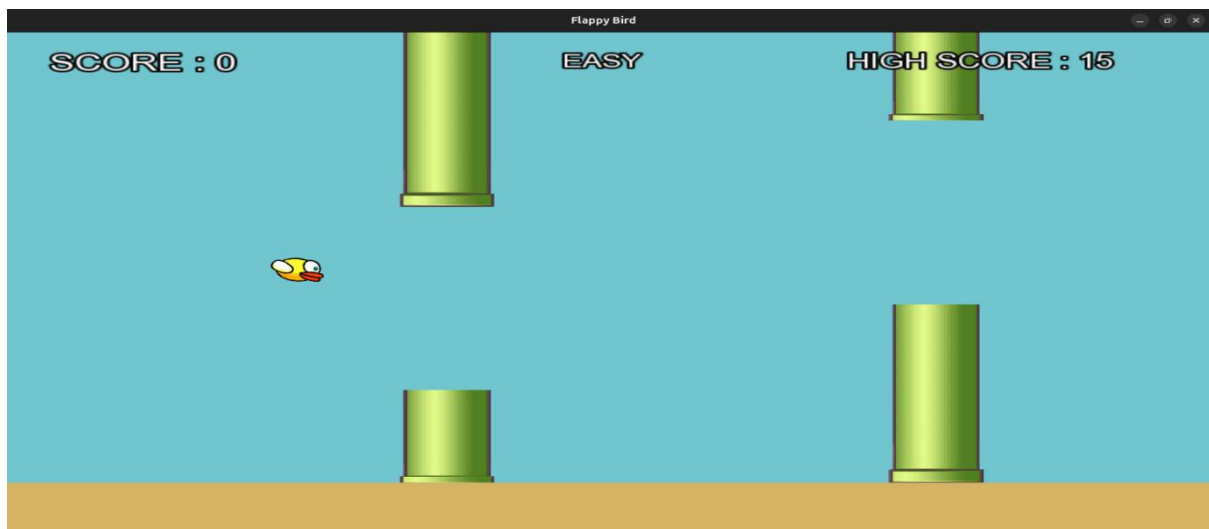


Figure 3: Mode Changed To Medium

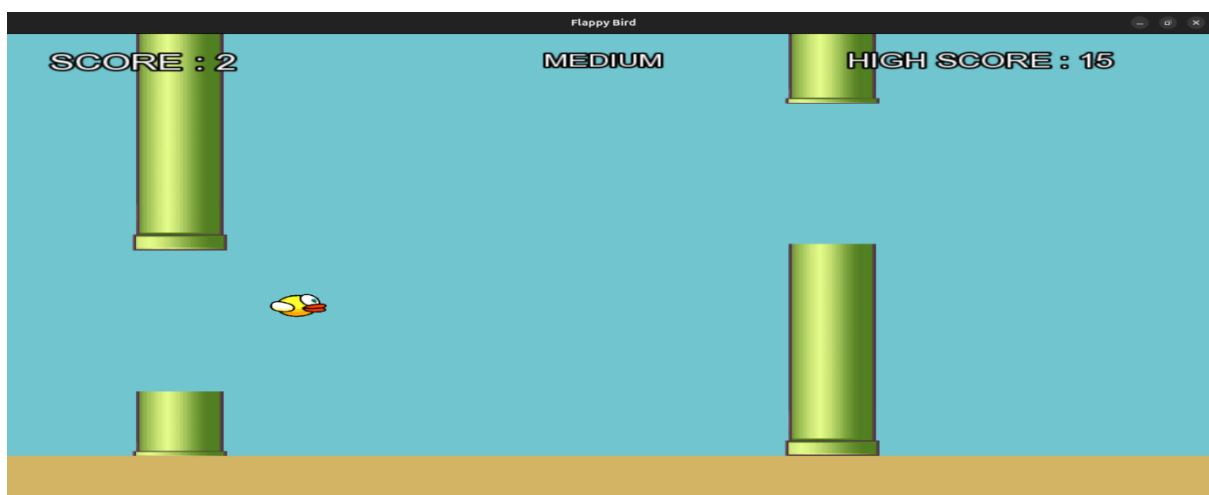


Figure 4: Hard Mode Selected For Game Play



Figure 5: Game Pause And Resume Feature Working (P & R Key Used)

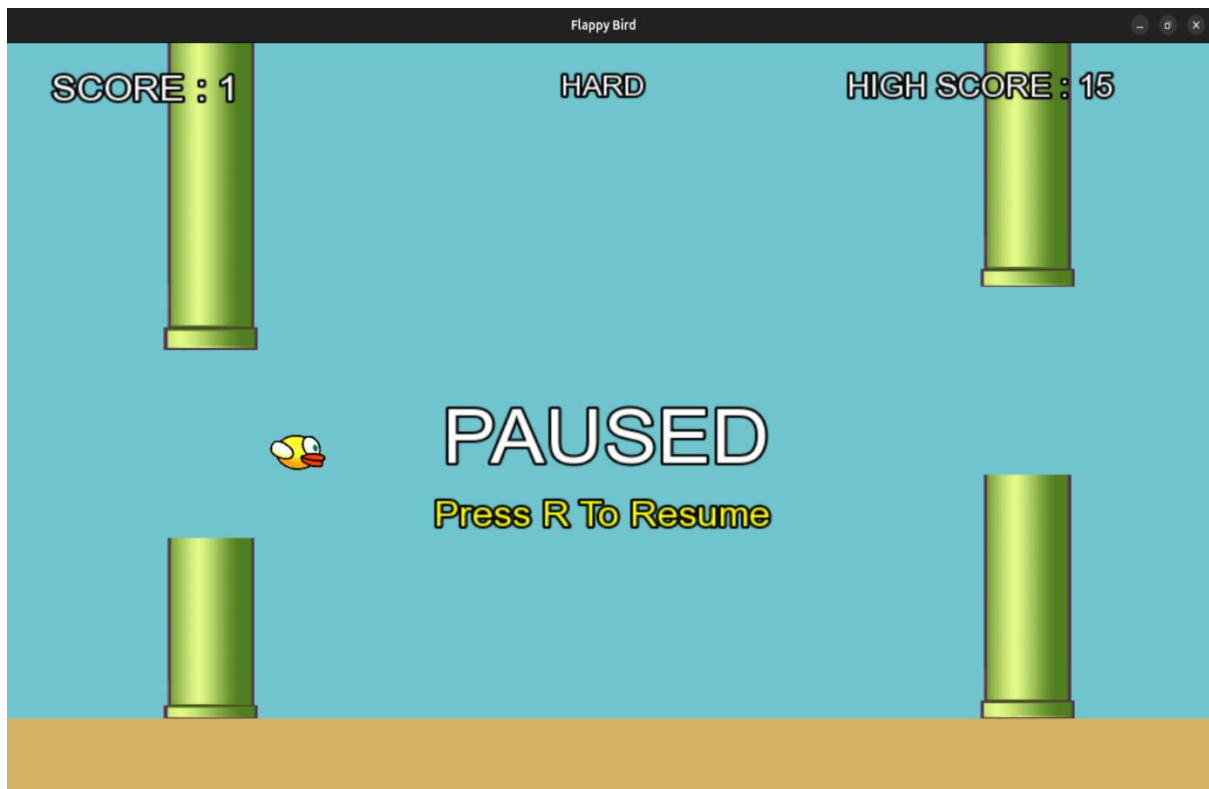
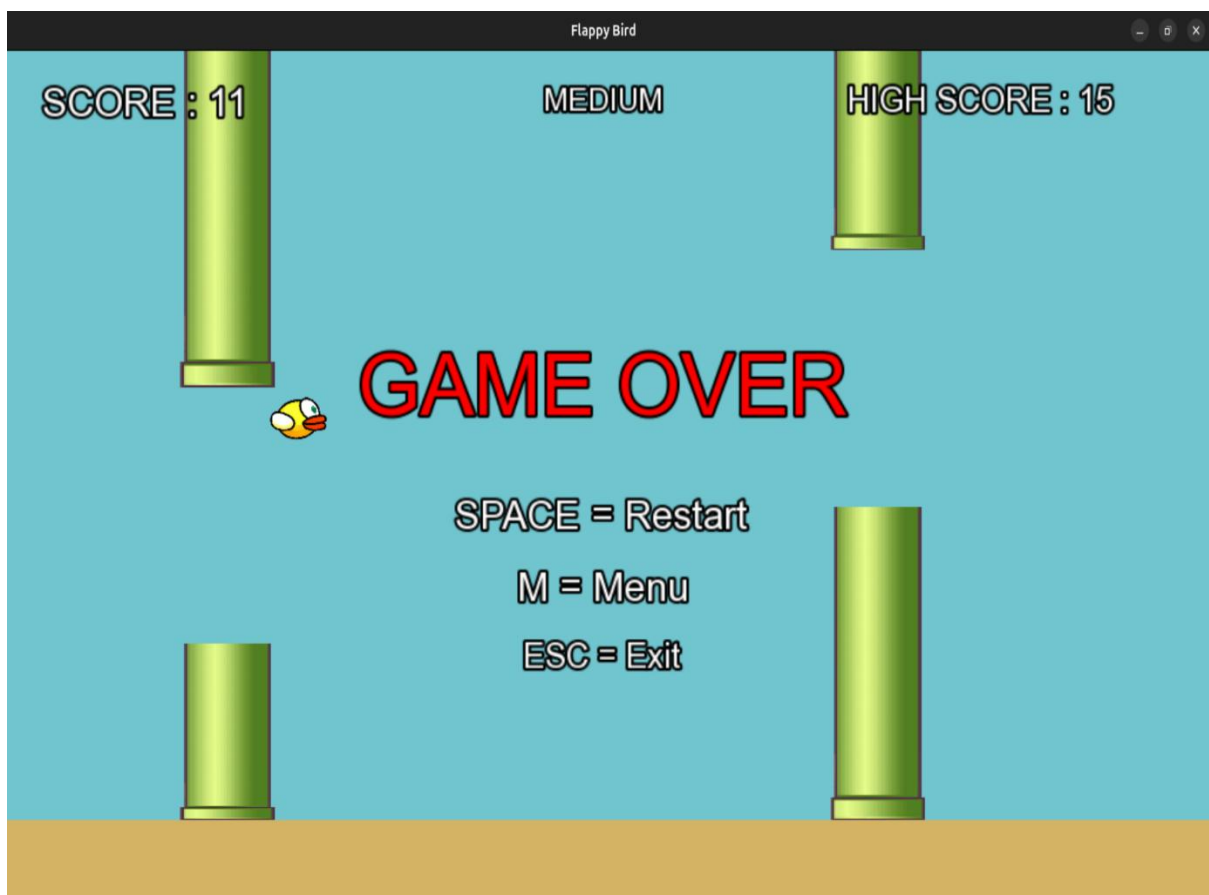


Figure 6: Collision Handled For Bird (Game Over Situation)



07. Logical Understanding

The logic of the program is based on a continuous real-time execution cycle that manages user input, object movement, collision handling, score updates, and graphical rendering. The game continuously runs inside a loop where all gameplay activities are updated repeatedly to maintain smooth and responsive interaction. During each iteration of the loop, the system processes keyboard events, updates the bird's position based on gravity, moves obstacles across the screen, checks for collisions, updates the score, and redraws all graphical elements. This continuous update-and-render mechanism forms the core logic of the game and ensures stable gameplay performance.

The logical flow of the program can be explained through the following points:

- Continuous game loop execution for handling real-time gameplay updates.
- Gravity affecting the bird's position and creating downward motion continuously.
- Player input (space key) used to move the bird upward during gameplay.
- Pipes moving horizontally across the screen at a constant speed.
- Random generation of obstacle gaps to increase gameplay difficulty.
- Collision detection using bounding boxes between the bird and pipes.
- Score increment when the bird successfully passes through obstacles.
- File handling to store and retrieve the highest score achieved by the player.
- Real-time rendering of graphics, score details, and game-over screens.
- Event handling for smooth keyboard interaction and responsive controls.

This project helps in understanding the flow of control, real-time execution, and logical implementation of game mechanics using C++ and SFML. It also demonstrates important concepts such as event-driven programming, frame-based updates, object management, collision handling, and multimedia rendering. Overall, the project provides practical exposure to real-time system behavior and interactive application development using object-oriented programming techniques.

08. Conclusion

In conclusion, this practical project was successfully completed using the C++ programming language and the SFML multimedia library. The development of the Flappy Bird-inspired game helped in strengthening the understanding of important programming concepts such as decision-making, iteration, functions, object-oriented programming, and real-time event handling. The project provided practical exposure to multimedia application development and demonstrated how programming logic and graphics libraries can be combined to create an interactive and engaging game. It also improved understanding of software structure, game mechanics, and event-driven programming techniques.

The implementation of the project strengthened understanding of several important concepts, including:

- Game loops and frame-based updates for smooth real-time gameplay.
- Physics simulation such as gravity and upward bird movement.
- Collision detection between the bird and obstacles.
- File handling for storing and retrieving high scores.
- Modular programming and class-based software design.
- Graphics rendering, animation, and event handling using SFML.
- Real-time object movement and score management techniques.

Overall, the project successfully achieved all intended objectives and significantly improved programming skills, logical thinking, debugging abilities, and problem-solving techniques. Testing and debugging under different gameplay conditions helped in improving the stability and performance of the application. This project also provided valuable practical experience in developing interactive multimedia applications and increased confidence in software development using C++ and SFML.

09. Future Scopes and Application

The future scope of this project includes several improvements that can enhance gameplay quality, graphics, and overall user experience. Although the current version of the Flappy Bird-inspired game successfully demonstrates the core concepts of game development using C++ and SFML, many advanced features can still be added to make the game more interactive and visually appealing. The project provides a strong foundation for implementing advanced multimedia functionalities, improving game mechanics, and understanding real-time software development concepts.

Some important future improvements and applications of this project include:

- Adding sound effects and background music to improve player engagement and game atmosphere
- Improving graphics quality, animations, and visual effects for a more attractive gaming experience
- Adding multiple difficulty levels to increase gameplay challenge and replay value
- Implementing menus such as start, pause, restart, and settings screens for better user interaction
- Using advanced physics concepts for more realistic bird movement and obstacle interaction
- Expanding the game into mobile or web-based versions for wider accessibility
- Adding different themes, backgrounds, and customizable characters for gameplay variety
- Introducing achievement systems, scoreboards, and online high-score sharing features
- Enhancing obstacle generation and gameplay mechanics using AI-based patterns

The project can also be extended into full-scale game development by adding advanced gameplay features, multiplayer support, and improved user interface designs. The concepts used in this project can further be applied in multimedia applications, animation systems, and interactive software development. Overall, the project serves as a strong base for learning advanced graphics programming, real-time rendering, and modern game development techniques using C++ and SFML.

References

- (1) Beginning C++ Game Programming by John Horton (Packt)
- (2) Object Oriented Programming with C++ (The McGraw-Hill)
- (3) SFML official documentation: <https://www.sfml-dev.org/documentation/>

Annexure

The following section presents the game source code along with explanations of the different components and functionalities implemented in the project.

Main.cpp:

Acts as the primary entry point of the Flappy Bird game and manages window initialization, asset loading, game states, event handling, bird physics, pipe generation, collision detection, scoring system, high-score file handling, rendering, audio management, and overall gameplay execution.

```
#include <SFML/Graphics.hpp>
#include <SFML/Audio.hpp>
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <ctime>
#include <cstdlib>
#include <algorithm>
#include "Bird.h"
#include "Pipe.h"
using namespace sf;

// Window Settings
const int WIN_W = 900;
const int WIN_H = 600;
const float GROUND_H = 60.f;
const float PIPE_SPAWN_X = WIN_W + 50.f;
const float PIPE_INTERVAL = 2.0f;

// Difficulty Settings
struct DifficultySettings
{
    std::string name;
    float gapSize;
    float pipeSpeed;
    bool oscillate;
};

// EASY / MEDIUM / HARD
const DifficultySettings DIFFICULTIES[3] =
{
    {"EASY", 220.f, 180.f, false},
    {"MEDIUM", 180.f, 240.f, false},
    {"HARD", 150.f, 300.f, true}};

// Game States
```

```

enum class GameState
{
    MENU,
    PLAYING,
    PAUSED,
    GAMEOVER
};

// Load High Score
int loadHighScore()
{
    std::ifstream file("HighScore.txt");
    int hs = 0;
    if (file.is_open())
    {
        std::string text;
        std::getline(file, text);
        std::string prefix = "High Score Till Now Is : ";
        if (text.find(prefix) != std::string::npos)
        {
            std::string number = text.substr(prefix.length());
            hs = std::stoi(number);
        }
        file.close();
    }
    return hs;
}

// Save High Score
void saveHighScore(int hs)
{
    std::ofstream file("HighScore.txt");
    if (file.is_open())
    {
        file << "High Score Till Now Is : " << hs;
        file.close();
    }
}

int main()
{
    std::srand(static_cast<unsigned>(std::time(nullptr)));

    // Create Window
    RenderWindow window(VideoMode(WIN_W, WIN_H), "Flappy Bird");
    window.setFramerateLimit(60);

    // VERY IMPORTANT FOR KEYBOARD INPUT
    window.requestFocus();
}

```

```

// Load Font
Font font;
if (!font.loadFromFile("Assets/arial.ttf"))
{
    std::cout << "Could not load font\n";
}

// Load Textures
Texture birdTex;
Texture pipeTex;

if (!birdTex.loadFromFile("Assets/Bird.png"))
{
    std::cout << "Could not load Bird texture\n";
}

if (!pipeTex.loadFromFile("Assets/Pipes.png"))
{
    std::cout << "Could not load Pipe texture\n";
}

// Ground
RectangleShape ground(Vector2f((float)WIN_W, GROUND_H));

ground.setFillColor(Color(210, 180, 100));
ground.setPosition(0.f, WIN_H - GROUND_H);

// Sounds
SoundBuffer flapBuf;
SoundBuffer hitBuf;
SoundBuffer scoreBuf;

// Correct sound file names
if (!flapBuf.loadFromFile("Assets/chop.wav"))
{
    std::cout << "Could not load flap sound\n";
}

if (!hitBuf.loadFromFile("Assets/death.wav"))
{
    std::cout << "Could not load hit sound\n";
}

if (!scoreBuf.loadFromFile("Assets/out_of_time.wav"))
{
    std::cout << "Could not load score sound\n";
}

Sound flapSound;
Sound hitSound;
Sound scoreSound;

```

```

flapSound.setBuffer(flapBuf);
hitSound.setBuffer(hitBuf);
scoreSound.setBuffer(scoreBuf);

flapSound.setVolume(60.f);
hitSound.setVolume(70.f);
scoreSound.setVolume(50.f);

// Background Music
Music bgMusic;
if (bgMusic.openFromFile("Assets/bgmusic.wav"))
{
    bgMusic.setLoop(true);
    bgMusic.setVolume(40.f);
    bgMusic.play();
}

// Game Variables
GameState state = GameState::MENU;

int diffIndex = 0;
int score = 0;

// Load High Score From File
int highScore = loadHighScore();

Bird bird(birdTex, WIN_W * 0.25f, WIN_H / 2.f);

std::vector<Pipe> pipes;
float pipeTimer = 0.f;

Clock clock;

// Helper Function for Text
auto makeText =
    [&](const std::string &str,
        unsigned size,
        Color color,
        float x,
        float y)
    {
        Text text(str, font, size);
        text.setFillColor(color);
        text.setOutlineColor(Color::Black);
        text.setOutlineThickness(2.f);
        FloatRect r = text.getLocalBounds();
        text.setOrigin(r.width / 2.f, r.height / 2.f);
        text.setPosition(x, y);
        return text;
    };

```

```

// Reset Game
auto resetGame = [&]()
{
    bird.reset(WIN_W * 0.25f, WIN_H / 2.f);
    pipes.clear();
    pipeTimer = 0.f;
    score = 0;
    state = GameState::PLAYING;
};

// GAME LOOP
while (window.isOpen())
{
    float dt = clock.restart().asSeconds();
    if (dt > 0.05f)
        dt = 0.05f;
    Event event;

    // Event Handling
    while (window.pollEvent(event))
    {
        if (event.type == Event::Closed)
        {
            window.close();
        }
    }

    // KEYBOARD INPUT
    if (event.type == Event::KeyPressed)
    {
        // ESC = EXIT GAME
        if (event.key.code == Keyboard::Escape)
        {
            window.close();
        }

        // ----- MENU -----
        if (state == GameState::MENU)
        {
            // EASY
            if (event.key.code == Keyboard::Num1 ||
                event.key.code == Keyboard::Numpad1)
            {
                diffIndex = 0;
                resetGame();
            }

            // MEDIUM
            if (event.key.code == Keyboard::Num2 ||
                event.key.code == Keyboard::Numpad2)
            {
                diffIndex = 1;
            }
        }
    }
}

```

```

        resetGame();
    }

    // HARD
    if (event.key.code == Keyboard::Num3 ||
        event.key.code == Keyboard::Numpad3)
    {
        diffIndex = 2;
        resetGame();
    }
}

// ----- PLAYING -----
else if (state == GameState::PLAYING)
{
    // Bird Jump
    if (event.key.code == Keyboard::Space)
    {
        bird.flap();
        flapSound.play();
    }

    // Pause Game
    if (event.key.code == Keyboard::P)
    {
        state = GameState::PAUSED;
    }
}

// ----- PAUSED -----
else if (state == GameState::PAUSED)
{
    // Resume Game
    if (event.key.code == Keyboard::R)
    {
        state = GameState::PLAYING;
    }
}

// ----- GAME OVER -----
else if (state == GameState::GAMEOVER)
{
    // Restart
    if (event.key.code == Keyboard::Space)
    {
        resetGame();
    }

    // Back To Menu
    if (event.key.code == Keyboard::M)
    {

```

```

        state = GameState::MENU;
    }
}
}

// Game Logic
if (state == GameState::PLAYING)
{
    const DifficultySettings &diff =
        DIFFICULTIES[diffIndex];

    // Bird Physics
    bird.update(dt);

    // Spawn Pipes
    pipeTimer += dt;

    if (pipeTimer >= PIPE_INTERVAL)
    {
        pipeTimer = 0.f;
        pipes.emplace_back(
            pipeTex,
            PIPE_SPAWN_X,
            (float)(WIN_H - GROUND_H),
            diff.gapSize,
            diff.pipeSpeed,
            diff.oscillate);
    }

    // Update Pipes
    for (auto &p : pipes)
    {
        p.update(dt, (float)(WIN_H - GROUND_H));
    }

    // Remove Offscreen Pipes
    pipes.erase(
        std::remove_if(
            pipes.begin(),
            pipes.end(),
            [](const Pipe &p)
            {
                return p.isOffScreen();
            }),
        pipes.end());

    // Score System
    for (auto &p : pipes)
    {
        if (p.checkScore(

```

```

        bird.sprite.getPosition().x))
    {
        score++;
        scoreSound.play();

        // Save New High Score
        if (score > highScore)
        {
            highScore = score;
            saveHighScore(highScore);
        }
    }
}

// Collision Detection
bool hit = false;

for (auto &p : pipes)
{
    if (p.collides(bird.getBounds()))
    {
        hit = true;
        break;
    }
}

// Top or Ground Collision
float birdTop =
    bird.getBounds().top;
float birdBottom =
    bird.getBounds().top +
    bird.getBounds().height;
if (birdTop <= 0.f ||
    birdBottom >= WIN_H - GROUND_H)
{
    hit = true;
}

// GAME OVER
if (hit)
{
    hitSound.play();
    state = GameState::GAMEOVER;
}
}

// DRAWING
window.clear(Color(113, 197, 207));

// Draw Ground
window.draw(ground);

```



```

// ----- MENU -----
if (state == GameState::MENU)
{
    window.draw(makeText(
        "FLAPPY BIRD",
        55,
        Color::Yellow,
        WIN_W / 2.f,
        140.f));

    window.draw(makeText(
        "Press 1 / 2 / 3",
        30,
        Color::White,
        WIN_W / 2.f,
        240.f));

    window.draw(makeText(
        "[1] EASY",
        28,
        Color::Green,
        WIN_W / 2.f,
        320.f));

    window.draw(makeText(
        "[2] MEDIUM",
        28,
        Color::Yellow,
        WIN_W / 2.f,
        380.f));

    window.draw(makeText(
        "[3] HARD",
        28,
        Color::Red,
        WIN_W / 2.f,
        440.f));

    window.draw(makeText(
        "HIGH SCORE : " +
        std::to_string(highScore),
        22,
        Color::White,
        WIN_W / 2.f,
        530.f));
}

```

```

// ----- GAMEPLAY -----
else
{
    // Draw Pipes
    for (auto &p : pipes)
    {
        p.draw(window);
    }

    // Draw Bird
    bird.draw(window);

    // Score
    window.draw(makeText(
        "SCORE : " +
        std::to_string(score),
        28,
        Color::White,
        110.f,
        30.f));

    // Difficulty Name
    window.draw(makeText(
        DIFFICULTIES[diffIndex].name,
        22,
        Color::White,
        WIN_W / 2.f,
        30.f));

    // High Score
    window.draw(makeText(
        "HIGH SCORE : " +
        std::to_string(highScore),
        24,
        Color::White,
        WIN_W - 170.f,
        30.f));

// ----- PAUSED -----
if (state == GameState::PAUSED)
{
    window.draw(makeText(
        "PAUSED",
        60,
        Color::White,
        WIN_W / 2.f,
        WIN_H / 2.f));

    window.draw(makeText(
        "Press R To Resume",
        28,

```

```

        Color::Yellow,
        WIN_W / 2.f,
        WIN_H / 2.f + 70.f));
    }

    // ----- GAME OVER -----
    if (state == GameState::GAMEOVER)
    {
        window.draw(makeText(
            "GAME OVER",
            60,
            Color::Red,
            WIN_W / 2.f,
            220.f));

        window.draw(makeText(
            "SPACE = Restart",
            28,
            Color::White,
            WIN_W / 2.f,
            320.f));

        window.draw(makeText(
            "M = Menu",
            28,
            Color::White,
            WIN_W / 2.f,
            370.f));

        window.draw(makeText(
            "ESC = Exit",
            24,
            Color::White,
            WIN_W / 2.f,
            420.f));
    }
}
window.display();
}
return 0;
}

```

Bird.h :

Defines the bird class responsible for sprite management, gravity-based movement, flap control, rotation animation, collision boundary handling, rendering, and position reset functionality for smooth real-time gameplay.

```
#pragma once
#include <SFML/Graphics.hpp>

class Bird
{
public:

    sf::Sprite sprite;
    float velocityY;
    float gravity;
    float flapStrength;

    static const float RENDER_W;
    static const float RENDER_H;

    Bird(sf::Texture &texture,
        float startX,
        float startY)

    {
        velocityY = 0.f;

        // FIXED VALUES
        gravity = 700.f;

        flapStrength = -320.f;
        sprite.setTexture(texture);

        sf::Vector2u texSize = texture.getSize();

        float scaleX = RENDER_W / texSize.x;
        float scaleY = RENDER_H / texSize.y;

        sprite.setScale(scaleX, scaleY);
        sprite.setOrigin(texSize.x / 2.f, texSize.y / 2.f);
        sprite.setPosition(startX, startY);
    }

    // Bird Jump
    void flap()
    {
        velocityY = flapStrength;
    }
}
```

```

// Update Bird
void update(float dt)
{
    velocityY += gravity * dt;
    sprite.move(
        0.f,
        velocityY * dt);

    // Rotation Animation
    float angle = velocityY * 0.05f;

    if (angle > 75.f)
        angle = 75.f;

    if (angle < -25.f)
        angle = -25.f;
    sprite.setRotation(angle);
}

// Collision Bounds
sf::FloatRect getBounds() const
{
    return sprite.getGlobalBounds();
}

// Draw Bird
void draw(sf::RenderWindow &window)
{
    window.draw(sprite);
}

// Reset Bird
void reset(float x, float y)
{
    sprite.setPosition(x, y);
    velocityY = 0.f;
    sprite.setRotation(0.f);
}
};

const float Bird::RENDER_W = 50.f;
const float Bird::RENDER_H = 40.f;

```

Pipe.h:

Implements the pipe obstacle class responsible for generating pipe pairs, handling horizontal movement, random gap positioning, vertical oscillation in hard mode, off-screen removal, score tracking, collision detection, and rendering of obstacles during gameplay.

```
#pragma once
#include <SFML/Graphics.hpp>
#include <cstdlib>
#include <cmath>

// Pipe Class
class Pipe
{
public:
    sf::RectangleShape topRect;
    sf::RectangleShape botRect;
    float x;
    float gapY;
    float gapSize;
    float speed;
    bool scored;
    bool oscillate;
    float oscillateTimer;
    float oscillateAmp;
    float oscillateSpeed;

    static const float PIPE_W;

    // Constructor
    Pipe(sf::Texture &pipeTexture,
        float startX,
        float playAreaH,
        float gap,
        float scrollSpeed,
        bool hardMode = false)
    {
        x = startX;
        gapSize = gap;
        speed = scrollSpeed;
        scored = false;
        oscillate = hardMode;
        oscillateTimer = 0.f;
        oscillateAmp = 55.f;
        oscillateSpeed = 2.0f;

        // Random gap position
        float margin = gapSize / 2.f + 70.f;
        float range = playAreaH - margin * 2.f;
```

```

    if (range < 1.f)
        range = 1.f;
    gapY =
        margin +
        (float)(std::rand() % (int)range);
    topRect.setTexture(&pipeTexture);
    botRect.setTexture(&pipeTexture);
    updateRects(playAreaH);
}

// Update pipe sizes
void updateRects(float playAreaH)
{
    float gapTop = gapY - gapSize / 2.f;
    float gapBottom = gapY + gapSize / 2.f;

    // Top Pipe
    float topH = gapTop;

    if (topH < 0.f)
        topH = 0.f;

    topRect.setSize(sf::Vector2f(PIPE_W, topH));
    topRect.setPosition(x - PIPE_W / 2.f, 0.f);

    // Bottom Pipe
    float botH =
        playAreaH - gapBottom;

    if (botH < 0.f)
        botH = 0.f;

    botRect.setSize(sf::Vector2f(PIPE_W, botH));

    botRect.setPosition(x - PIPE_W / 2.f, gapBottom);
}

// Update movement
void update(float dt, float playAreaH)
{
    // Move left
    x -= speed * dt;

    // Hard mode oscillation
    if (oscillate)
    {
        oscillateTimer += dt;
        float delta =
            oscillateAmp *
            std::sin(
                oscillateSpeed *

```

```

        oscillateTimer) *
        dt;
        gapY += delta;
        float margin = gapSize / 2.f + 70.f;

        if (gapY < margin)
            gapY = margin;

        if (gapY > playAreaH - margin)
            gapY = playAreaH - margin;
    }
    updateRects(playAreaH);
}

// Remove pipe off screen
bool isOffScreen() const
{
    return (x + PIPE_W / 2.f) < 0.f;
}

// Score check
bool checkScore(float birdX)
{
    if (!scored && birdX > x)
    {
        scored = true;
        return true;
    }
    return false;
}

// Collision check
bool collides(sf::FloatRect birdBounds) const
{
    return birdBounds.intersects(topRect.getGlobalBounds()) ||
        birdBounds.intersects(botRect.getGlobalBounds());
}

// Draw pipes
void draw(sf::RenderWindow &window)
{
    window.draw(topRect);
    window.draw(botRect);
}
};

// Static Width
const float Pipe::PIPE_W = 70.f;

```